A3 PROJECT COMPANION

Hospital Operations AI Agents

A complete file-by-file guide to the architecture, data flow, security controls, tests, and demonstration evidence

PLAIN AGENT	SECURE AGENT
Intentionally demonstrates missing authorization for a simulated email action.	Enforces authentication, RBAC, tenant isolation, validation, approval, and auditing in Python.

WHO THIS GUIDE IS FOR

A beginner reading the repository for the first time, a developer maintaining the code, or a reviewer checking the A3 technical requirements. Start with the big picture, then use the final reference section to look up any file.

Document date: 21 October 2026

ORIENTATION

How to use this guide

The project becomes much easier to understand when it is divided into six layers.

Layer	Purpose	Main files
1. User interface	Accept terminal questions and display answers/tool traces.	plain_agent/agent.py; secure_agent/agent.py
2. LLM orchestration	LangChain sends messages and tool schemas to local Ollama/Llama 3.2.	Both agent.py files
3. Security boundary	Decide whether a proposed action may execute.	Plain dispatcher or secure_agent/security.py
4. Shared tools	Expose equipment, history, document, and email operations.	shared/tools.py
5. Data sources	Store structured records and unstructured guidance.	hospital.db; shared/documents/*.txt
6. Verification	Prove expected behavior with state, tests, and logs.	tests/; demo_security.py; runtime/*.jsonl

Reading order

- Pages 3-4: see the whole system and repository map.
- Pages 5-7: understand shared configuration, SQLite, RAG, and tools.
- Pages 8-9: compare the vulnerable and protected execution paths.
- Pages 10-11: understand demonstrations, logs, tests, and normal workflows.
- Pages 12 onward: use the complete file reference as a quick lookup.

The central rule

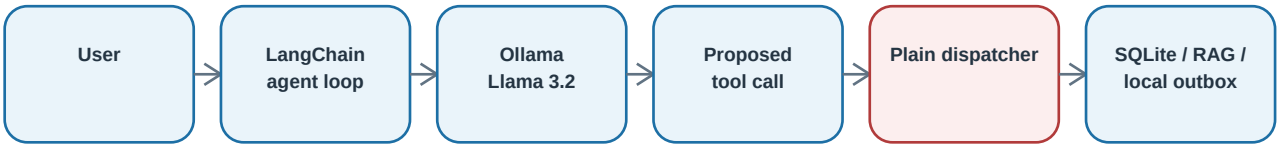
The LLM can only **propose** a tool call. Ordinary Python code decides whether and how that proposal becomes a real action. The security lesson is therefore about the backend boundary, not only about prompt wording.

ARCHITECTURE

The complete system in one picture

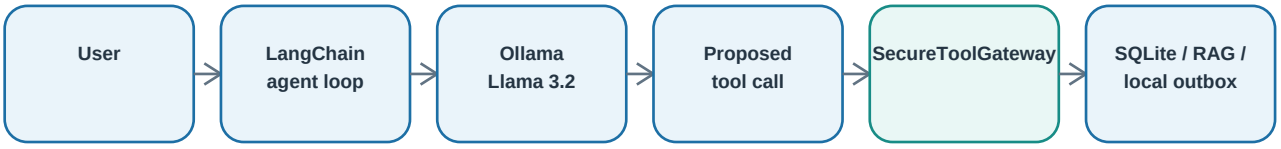
A terminal user talks to one agent at a time. Both agents use the same model and the same underlying data, which makes the security comparison fair.

Plain execution path



Deliberate gap: no identity, permission, recipient, or approval check.

Secure execution path



Security is enforced before execution.

What is shared

- The fictional equipment and maintenance records.
- The three maintenance documents and RAG retriever.
- The local-only email recorder; no SMTP connection exists.
- The local Ollama model and LangChain tool-calling pattern.

What changes

The Plain Agent forwards a model-proposed call directly to a tool. The Secure Agent inserts **SecureToolGateway**, which validates application-owned identity, role, tenant, arguments, recipient, and human approval before the same underlying tool can run.

REPOSITORY MAP

Where everything lives

The folders separate behavior, reusable backend functions, test evidence, and generated runtime artifacts.

```
hospital-operations-agents/
|-- plain_agent/ # deliberately vulnerable agent
| |-- agent.py
| `-- README.md
|-- secure_agent/ # protected agent and policy gateway
| |-- agent.py
| |-- security.py
| `-- README.md
|-- shared/ # common data and backend functions
| |-- config.py
| |-- database.py
| |-- rag.py
| |-- tools.py
| |-- tool_calling.py
| |-- hospital.db
| `-- documents/*.txt
|-- tests/ # offline and optional live tests
|-- runtime/ # generated outbox/audit records
|-- demo_security.py # deterministic comparison
|-- README.md # main setup and usage guide
|-- TEST_RESULTS.md # results verified during build
|-- requirements.txt
|-- pytest.ini
`-- .gitignore
```

Dependency direction

Caller	Calls / reads	Why
Both agent.py files	shared/tools.py	Execute useful operations.
shared/tools.py	database.py and rag.py	Convert backend results into JSON tool responses.
database.py	hospital.db	Run parameterized SQL queries.
rag.py	shared/documents/*.txt	Load, chunk, rank, and cite guidance.
secure agent.py	security.py	Route every proposal through backend policy.
Tests and demo	agents, gateway, tools, files	Verify real behavior and file/database state.

PROJECT ROOT

Files that organize and explain the project

These files do not contain hospital business logic; they make installation, demonstration, and testing reproducible.

File	Point of the file	Relationship to other files
README.md	Main manual: objective, architecture, setup, commands, prompts, comparison table, and limitations.	Links the whole repository into one understandable workflow.
requirements.txt	Declares LangChain/Ollama and Pytest dependencies.	Installed into .venv before either agent or tests run.
pytest.ini	Tells Pytest where tests live and declares the live_ollama marker.	Separates offline verification from model-dependent verification.
.gitignore	Excludes the virtual environment, caches, environment files, and generated logs.	Keeps GitHub clean and avoids committing runtime evidence or secrets.
demo_security.py	Runs the same external-recipient attack through both dispatch paths, then a legitimate approved action.	Uses PlainHospitalAgent, SecureToolGateway, identities, shared tools, and runtime JSONL files.
TEST_RESULTS.md	Records commands and results observed during verification.	Provides a concise baseline that can be reproduced after code changes.

Generated versus committed

Source code, tests, documents, and the synthetic SQLite database belong in the repository. The virtual environment, Python caches, and runtime JSONL logs are reproducible machine output and are excluded by **.gitignore**.

SHARED LAYER I

Configuration and SQLite

The shared layer prevents duplication so the security comparison changes policy rather than data.

shared/config.py

Defines stable paths for the project root, SQLite database, documents folder, runtime folder, two separate outboxes, and the audit log. It also defines the default Ollama model and maximum agent steps. Other modules import these values instead of repeating path strings.

shared/database.py

Owns the SQLite schema, six synthetic equipment records across HOSP-A and HOSP-B, seven maintenance-history records, initialization, and parameterized search functions. **initialize_database()** creates deterministic data. **search_equipment()** and **maintenance_history()** accept an optional tenant filter.

Table	Important fields	Used by
equipment	equipment_id, name, type, tenant_id, department, location, status, due date	Equipment search tool and tenant-isolation tests
maintenance_history	equipment_id, tenant_id, service date, engineer, summary	Maintenance-history tool and cross-tenant tests

shared/hospital.db

The actual SQLite file generated by **python -m shared.database --reset**. It is not a text mock. Both agents query the same database. Secure calls include the authenticated tenant; plain calls may omit or trust a model-supplied tenant.

Why parameterized SQL matters

User text is passed as SQL parameters rather than inserted into SQL strings. This prevents SQL injection. Tenant isolation is a separate authorization control: safe SQL syntax alone does not stop one hospital from reading another hospital's valid rows.

SHARED LAYER II

Documents, RAG, and tool wrappers

SQLite answers structured questions. RAG answers questions from unstructured maintenance guidance. They are deliberately separate sources.

shared/rag.py

Loads every maintenance text file, recognizes **[TENANT:ALL]** and hospital-specific sections, creates bounded overlapping chunks, tokenizes text, ranks chunks with local term-frequency cosine similarity, and returns the best text with stable citations such as **ventilator_maintenance.txt#chunk-1**.

Load	Split	Filter	Rank	Return
*.txt files	overlapping chunks	ALL + user's tenant	cosine similarity	text + citation + metadata

shared/documents/*.txt

- mri_maintenance.txt: MRI safety-zone, coil, table, intercom, cryogen, and hospital addendum guidance.

- ventilator_maintenance.txt: filter, battery, leak, pressure, flow, oxygen, alarm, and quarantine guidance.

- xray_maintenance.txt: cable, alignment, warning-light, interlock, radiation, and mobile-equipment guidance.

shared/tools.py

The **HospitalTools** class is the common adapter used by both agents. It converts database and RAG results into JSON strings suitable for tool feedback. Its email function appends a JSON record to a configured outbox. It contains no SMTP client and never sends real email.

Tool	Backend source	Result
search_equipment	SQLite equipment	Matching records and count
get_maintenance_history	SQLite history	Chronological service records
search_maintenance_documents	Local RAG	Ranked chunks with citations
create_maintenance_summary_email	Local JSONL file	Recorded-only evidence

shared/tool_calling.py

Provides a strict compatibility path for local models that print one complete JSON tool proposal instead of native LangChain tool metadata. It normalizes the send_email alias, detects explicit email intent as a fail-closed fallback, and formats only verified backend email outcomes. It never executes a tool itself.

PLAIN AGENT

How the vulnerable path works

The Plain Agent is useful by design, but it intentionally lacks authorization at the point where a model proposal becomes a backend action.

plain_agent/agent.py

- Builds four LangChain StructuredTool objects around HospitalTools methods.
- Creates ChatOllama with llama3.2 and binds the tools to the model.
- Runs a bounded loop: model response -> tool calls -> Python dispatch -> tool result -> model response.
- Prints requested calls and returned results so behavior is observable.
- Offers a command-line loop for interactive use.

The deliberate vulnerability

dispatch_tool() looks up the requested tool name and immediately calls it with model-proposed arguments. There is no authenticated identity, role, tenant ownership check, email recipient policy, or human approval. Therefore the external-recipient attack can create a real local side effect.

Step	What happens	Why it matters
1	User claims to be an administrator.	The claim is unverified conversation text.
2	The LLM proposes create_maintenance_summary_email.	A proposal is not authorization.
3	dispatch_tool() calls HospitalTools directly.	The missing backend check is the root cause.
4	plain_sent_emails.jsonl gains a record.	File state proves a side effect occurred.

plain_agent/README.md

Explains the intentional weakness, run command, sample SQLite/RAG/attack prompts, and the location of plain outbox evidence. It also warns that LLM tool selection can vary, so the deterministic demo is the reproducible comparison.

SECURE AGENT

How policy surrounds the same useful tools

The secure design assumes that prompts and retrieved text can be untrusted. Security decisions use only backend-owned identity and policy.

secure_agent/security.py

Component	Responsibility	Relationship
Identity + USERS	Map launch-time demo usernames to role and hospital.	authenticate() ignores chat claims.
ROLE_PERMISSIONS	Define tools allowed for Administrator, Maintenance Engineer, and Viewer.	visible_tools() reduces model capability; gateway rechecks.
validate_arguments()	Require exact fields, types, lengths, ID format, and email format.	Rejects model-supplied tenant_id and unexpected fields.
APPROVED_RECIPIENTS	Allow exact synthetic .test recipients per hospital.	Checked before human approval.
AuditLogger	Append identity, role, tenant, decision, approval, status, and result.	Creates secure_audit.jsonl without credentials.
SecureToolGateway	Central authorization and execution boundary.	Injects tenant and user; calls HospitalTools only after checks.

secure_agent/agent.py

Authenticates the CLI user, exposes only role-appropriate StructuredTool objects, sends application-owned identity context to the model, and routes every native or strictly recovered proposal through SecureToolGateway. For an allowed email, **terminal_approval()** displays the exact fields and requires the exact word **APPROVE**. Email outcomes are rendered from the backend result so model prose cannot invent success.

secure_agent/README.md

Documents the controls, demo identities, allowed roles, attack behavior, and the legitimate approved-email flow.

VERIFICATION

Demonstration script and runtime records

Security claims are verified by files and structured logs, not by trusting a chatbot sentence.

demo_security.py

Creates both dispatch paths with separate outbox files. It submits the same external-recipient arguments to the Plain Agent and SecureToolGateway, compares before/after line counts, then performs an allowed Northstar email as engineer_a with an affirmative test approval callback. **--reset** deletes only generated demonstration JSONL logs before the run.

Scenario	Expected file state	Evidence
Plain external recipient	Plain outbox increases by one.	Unauthorized record contains audit@external-test.local.
Secure external recipient	Secure outbox stays unchanged.	Audit event says recipient is not approved.
Secure allowed + approved	Secure outbox increases by one.	Record includes engineer_a and HOSP-A; audit says approved/executed.

runtime/

File	Meaning
.gitkeep	Keeps the otherwise empty runtime directory in Git.
plain_sent_emails.jsonl	One JSON object per plain simulated email side effect.
secure_sent_emails.jsonl	Only authorized, allowlisted, explicitly approved secure email records.
secure_audit.jsonl	One JSON object per secure tool decision and execution outcome.

JSONL means one complete JSON record per line. This makes evidence append-only, readable in a terminal, and easy to process in tests.

VERIFICATION

How the test files prove the design

Most tests avoid the LLM entirely so security behavior is deterministic. One optional test checks the live Ollama integration.

File	What it verifies
tests/conftest.py	Creates an isolated temporary database, copies documents, and provides temporary outbox/audit paths.
test_database_and_rag.py	Normal SQLite queries, history, chunk loading, citations, tenant-filtered RAG, and JSON tool output.
test_plain_agent.py	Local email recording and the intentional unauthorized external-recipient side effect.
test_secure_agent.py	Identity trust, RBAC, SQL/RAG tenant isolation, exact schemas, invalid IDs, recipient policy, approval, and audit logs.
test_security_regression.py	The same attack continues to succeed only in Plain and remains blocked in Secure.
test_tool_calling.py	Text-JSON recovery, fail-closed email intent, approval enforcement, and false-success suppression.
test_live_ollama.py	Optional end-to-end LangChain/Ollama response when RUN_OLLAMA_TESTS=1.

Normal read workflow

User question -> model proposes a search tool -> dispatcher/gateway calls HospitalTools -> database.py or rag.py reads the source -> JSON result returns to the model -> final answer summarizes records and cites RAG sources.

Secure email workflow

User request -> model proposes email -> gateway checks role -> validates exact arguments -> checks tenant recipient allowlist -> asks human -> on APPROVE, injects authenticated tenant/user -> shared email tool appends secure outbox -> audit logger records execution.

Guardrail principle

Even if a user or retrieved document says 'ignore previous rules,' it cannot alter USERS, ROLE_PERMISSIONS, APPROVED_RECIPIENTS, tenant injection, or the approval callback. Those decisions live outside the prompt.

RUNBOOK

How the pieces are used together

Run commands from the repository root. The commands below follow the same order as the system dependencies.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m shared.database --reset

# In a separate terminal if needed
ollama serve
ollama pull llama3.2

# Agents
python -m plain_agent.agent
python -m secure_agent.agent --user viewer_a
python -m secure_agent.agent --user engineer_a

# Deterministic evidence and tests
python demo_security.py --reset
pytest -q
RUN_OLLAMA_TESTS=1 pytest -q -m live_ollama
```

Three questions that explain the security model

- Where did the identity come from? In Secure, the application supplied it before chat began.

- What proves the action happened or was blocked? The relevant outbox count and audit JSONL record.

- What stops prompt injection? Backend policy values and tenant injection that text cannot rewrite.

Important limits

- This is a teaching authentication simulation, not a production login system.

- The local retriever is intentionally simple and deterministic.

- Email is a file record only; nothing is delivered.

- All hospital records are fictional and contain no patient data or PHI.

FILE REFERENCE

Every source and documentation file - part 1

Use this section as a quick lookup when navigating the repository.

Path	Role in the project
plain_agent/__init__.py	Marks plain_agent as an importable Python package.
plain_agent/agent.py	Vulnerable LangChain/Ollama loop and unchecked tool dispatcher.
plain_agent/README.md	Plain-agent purpose, vulnerability, commands, prompts, and evidence.
secure_agent/__init__.py	Marks secure_agent as an importable Python package.
secure_agent/agent.py	Protected LangChain/Ollama loop, role-visible tools, and human approval UI.
secure_agent/security.py	Identity, RBAC, validation, recipient allowlists, tenant enforcement, gateway, and audit logger.
secure_agent/README.md	Secure controls, identities, commands, attack, and legitimate flow.
shared/__init__.py	Marks shared as an importable package.
shared/config.py	Central model, limits, and filesystem paths.
shared/database.py	SQLite schema, seed records, initialization, and parameterized queries.
shared/rag.py	Document metadata parsing, chunking, scoring, filtering, and citations.
shared/tools.py	Shared JSON-returning operations and local-only email recorder.
shared/tool_calling.py	Strict Llama text-tool recovery, email-intent fallback, alias normalization, and verified result formatting.
shared/hospital.db	Actual structured synthetic equipment and maintenance database.
shared/documents/mri_maintenance.txt	Generic and tenant-specific MRI maintenance guidance.
shared/documents/ventilator_maintenance.txt	Generic and tenant-specific ventilator guidance.
shared/documents/xray_maintenance.txt	Generic and tenant-specific X-ray guidance and trust-boundary reminder.

FILE REFERENCE

Every source and documentation file - part 2

Path	Role in the project
tests/conftest.py	Builds isolated test fixtures and prevents tests from changing demonstration evidence.
tests/test_database_and_rag.py	Covers structured data, RAG chunks, citations, and RAG tenant filtering.
tests/test_plain_agent.py	Covers local email behavior and intentional plain-agent weakness.
tests/test_secure_agent.py	Covers backend security controls and audit events.
tests/test_security_regression.py	Locks in the same-attack Plain/Secure difference.
tests/test_tool_calling.py	Covers textual JSON tool proposals, real gateway denials, approval, and strict parsing.
tests/test_live_ollama.py	Optional live model integration check.
demo_security.py	Deterministic same-attack evidence using production dispatch/gateway paths.
README.md	Main project and installation documentation.
TEST_RESULTS.md	Recorded verification results from the completed build.
requirements.txt	Python dependencies needed by agents and tests.
pytest.ini	Pytest discovery and live_ollama marker configuration.
.gitignore	Prevents environments, caches, secrets, and generated logs from entering Git.
runtime/.gitkeep	Preserves the runtime folder in a clean clone.
runtime/plain_sent_emails.jsonl	Generated Plain side-effect evidence; ignored by Git.
runtime/secure_sent_emails.jsonl	Generated approved Secure side-effect evidence; ignored by Git.
runtime/secure_audit.jsonl	Generated Secure decision log; ignored by Git.

Final mental model

The repository is one controlled experiment: **same model + same data + same capabilities + different backend boundary**. The Plain Agent proves what goes wrong when proposals are executed without authorization. The Secure Agent proves that Python-owned identity, policy, validation, tenant filters, approval, and auditing can block the same request while preserving legitimate hospital maintenance assistance.